MMA3001 Project Report

Automated Detection of Pork Rasher Packaging Errors and Meat Quality

1.0 Engineering Problem

Imagine working as an automation engineer in a modern food processing plant. On the factory floor fresh pork rashers are placed into plastic trays and sealed by packaging machines. The trays zoom past on a conveyer belt at around 1.5 meters per second, which is around 100 trays every minute and more than one tray per second (Bison Convey, n.d.). Workers used to stand by the belt and inspect the meat trays, but with the speed at which the packages go by it is impossible to keep up and reliably spot a tiny tear in the packaging or a small brown oxidation patch on the meat. In continuous high-speed packaging, three main defects routinely occur.

- **Packaging Film Punctures and Tears:** Tearing or puncture of the top clear film caused by sharp bone fragments, cartilage, or slipping robotic grippers, which immediately destroys the gas barrier between atmosphere and the internal vacuum.
- **Seal Flange Perimeter Contamination:** Meat fibres, moisture droplets, or grease trapped within the $10\ mm$ heat-seal border around the edge of the tray, forming pathways through which air slowly leaks.
- **Meat Quality Degradation and Discoloration:** Premature chemical spoilage that oxidizes healthy red meat into a dull grey-brown colour, signalling packaging seal failure or temperature abuse.

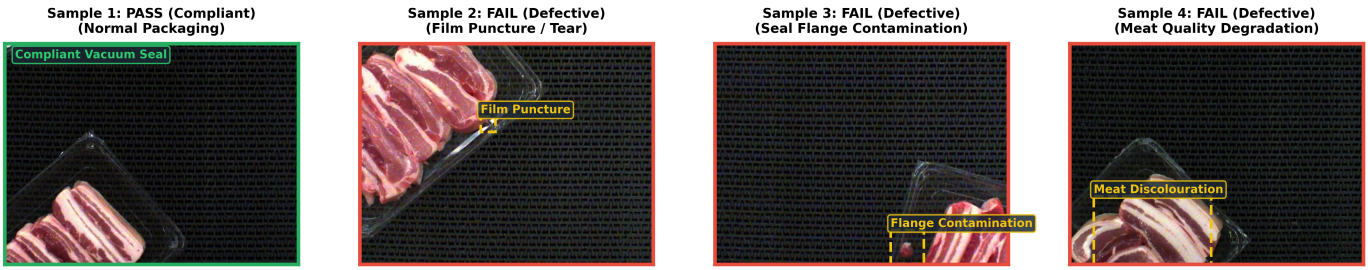


Figure 1.1: High-resolution industrial inspection frames **(Hello, 2025)**, *illustrating compliant packaging (Pass), film puncture tear, flange seal contamination, and meat discoloration defect states.*

These errors can cause health and safety problems to buyers, promote unnecessary food waste, and ruin the reputation of the company which distributes these meats. A false negative where a breached or spoiled tray is missed during inspection can cause mandatory recalls and cost the company millions of dollars to amend. Conversely, a false positive only incurs a very small cost to the company, as each tray discarded is only worth a few cents.

This is where machine learning enters the picture, as a camera can provide a computer program with images where it can detect any problems with the meat and determine whether it passes or fails inspection immediately. This can feed information to high-speed pneumatic sorting rejectors, which must act fast enough to push only the failed packages off the conveyor belt. Furthermore, as meat trays are sealed at near zero pressure, a computational solution can determine the defect sizing and use this to evaluate pressure decay to quantitatively predict

vacuum loss and shelf-life depletion. This could allow less food waste by being able to determine if the meat will last long enough to be distributed and sold at supermarkets, or if it should be thrown out before it reaches shelves.

A limitation associated with this approach is that single trays are only visible from an orthogonal overhead camera, so the underside, walls, or secondary packaging are out of scope and cannot be checked for packaging defects. Additionally, the computer must assume controlled lighting, and performance can degrade under severe glare from the packaging film, lens condensation, or uncalibrated shifts in ambient lighting. As images on capture a single moment, this does not account for any downstream issues that occur further down the conveyor belt or in transport which can cause the pork rasher packaging to no longer be deemed safe.

2.0 Inputs and Outputs

2.1 Summary of Inputs and Outputs

The automated quality inspection system operates on high resolution overhead digital image frames alongside real time factory sensory inputs. To simulate industrial packaging conditions, the pipeline utilises a real pork rasher image dataset ($1{,}955$ raw images augmented to $3{,}549$ standardised 720×540 frames). To prevent Google Colab memory exhaustion during batch training, a statistically balanced subset of 480 images (400 training, 80 hold-out testing) is used. These visual inputs, alongside ambient pressure, package vacuum levels, and conveyor kinematics, drive the defect segmentation, physical air leak simulation, and pass or fail classification models detailed in Table 2.1 and Table 2.2 below.

Table 2.1: Specification and intended use of each input

Input	Specification	Physical Meaning	Intended Use
Photo of the Tray (Camera Image)	540×720 pixels, RGB image	A top-down photo taken by an overhead camera under flash lighting as the tray moves along the conveyor belt.	The main visual picture the computer scans to check for ripped plastic, dirty seals, or bad meat.
Colour and Brightness (Pixel Values)	Numbers from 0 to 255 for each colour dot	How bright, dark, red, or white each pixel in the photo is.	Allows the computer to tell fresh red meat apart from rotten brown meat, white fat, or dark ripped holes.
Camera Zoom Scale ($mm/pixel$)	$1\ px = 0.33mm$ (or $3\ px/mm$)	How zoomed in the camera is, it links pixels to real-world metric measurements.	Allows the computer to measure real world hole sizes (mm^2) instead of just counting pixels.
Tray Dimensions	$240\ mm$ long x $180\ mm$ wide (total area = $43200\ mm^2$)	The real physical size of the plastic container holding the bacon rashers.	Tells the computer where the tray begins and ends and catches errors if something is the wrong size.
Sealed Border Width (Flange Edge)	$10\ mm$ wide plastic rim around the tray	The flat border where the lid plastic gets melted onto the container to trap the vacuum.	Directs the computer to inspect this border for trapped meat or grease that can cause air leaks.

Starting Vacuum Inside Tray	$20\ kPa$	How tightly the air was sucked out of the package before sealing to keep food fresh.	The starting number for math equations that simulate how fast air will rush in if the plastic rips.
Room Air Pressure Outside	$101.3\ kPa$ (Young, 2017)	The everyday air pressure in the factory room outside of the package.	Tells the physics equation how hard the outside room air is trying to force its way into a hole.
Air Leak Speed Factor (k)	$k = 0.8/s$	A measure of how easily and quickly air can rush through a hole.	Used in the physics equation to calculate vacuum loss inside a damaged package each second.
Conveyor Belt Speed	1 to $1.5\ m/s$ (60 to 100 trays every minute)	How fast the trays are zooming past on the factory belt.	Sets a $0.6\ s$ deadline for the computer to finish inspecting each tray before the next arrives.
Minimum Hole Size to Reject	$16.6\ mm^2$ (~150 picture pixels)	The smallest rip big enough to ruin the package or break the seal.	Prevents the robot from stopping the factory line for harmless tiny dust specks or minor glare.
Practice Training Photos	480 verified factory photos (400 training, 80 testing)	A collection of real packaging photos showing good trays, tears, bad seals, and spoiled meat.	Teaches the smart AI model what a bad tray looks like so it can spot defects automatically on its own.

Table 2.2: Specification and intended use of each output

Output	Specification	Physical Meaning	Intended Use
Pass or Fail Decision	0 (Pass) or 1 (Fail)	The final verdict on whether the tray is good to sell or defective.	Sends an instant electronic trigger to a robot pusher arm to push bad trays off the belt into a reject bin.
Defect Confidence Score	Percentage from 0% to 100%	How confident the AI is that the package has a real tear, bad seal, or spoiled meat.	Helps factory managers decide how strict the line is.
Defect Outline Map (Visual Mask)	540×720 black-and-white grid highlighting defect spots	A highlight overlay showing the exact location and shape of the rip, hole, or trapped meat.	Displays on a TV screen for factory workers to see the issue and allows calculation of the defect area.
Measured Defect Size (Hole Area)	Real physical area measured in square millimetres (mm^2)	The true real-world size of the tear or grease leak.	Tells workers how severe the damage is, helping sort minor and large damage.
Air Leak Pressure Curve	Graph tracking pressure rising from $20\ kPa$ to $101.3\ kPa$ over 10 seconds	A physics calculation showing how fast air rushes in and collapses the vacuum inside the pack.	Predicts if the meat will spoil before reaching supermarket shelves.
Time Until Freshness is Lost	Time in seconds	How many seconds of leaking it takes before the food completely loses its freshness protection.	Informs food safety managers how quickly punctured trays must be dealt with before bacteria can multiply.
Computer Check Speed	1.6 to 8.7 milliseconds per photo (115 to 620 photos per second)	The tiny fraction of a second the computer takes to check one tray.	Proves the computer is faster than the conveyor belt, guaranteeing no slowdowns.

Error Warning Codes	Text alerts	Specific alerts detailing if the camera flash broke, the lens was blinded by glare, or water splashed the glass.	Safely diverts unreadable trays to a manual check lane without crashing the automated system.

2.2 Handling Invalid, Missing, or Unsupported Inputs

To guarantee engineering robustness, an input validator enforces four strict guardrails to ensure inputs are valid before any inference occurs. If a frame violates these rules, the system immediately raises an error with a structured diagnostic code, diverts the suspect tray to a secondary manual inspection bin, and flags an alert on the operator human-machine interface without crashing.

- **ERR_DIM / ERR_DTYPE**: Rejects image frame tensors deviating from $(540 \times 720 \times 3)$ or non-uint8 data types.
- **ERR_UNDEREXPOSED**: Triggered when frame mean intensity is less than 15. Indicates camera flash power failure or a closed lens cap.
- **ERR_OVEREXPOSED**: Triggered when frame mean intensity is above 245. Identifies severe optical glare or reflection off surrounding surfaces.
- **ERR_BLANK / ERR_OUT_OF_DOMAIN**: Rejects frames with variance less than 15 (solid black or white frames) or spatial gradients more than 65 (uncorrelated random transmission static).

3.0 Computational Solution

3.1 The Computational Method

To tackle the inspection challenge on the factory line, the computational workflow connects dual classification models, 2D numerical defect sizing, and dynamic air leak modelling into a single seamless pipeline. The first line of defence is a baseline colour rule detector, which inspects the tray image by checking the flat outer flange rim for abnormal red discoloration caused by meat or blood trapped in the heat seal, scanning the cavity for dark torn holes where the underlying tray is exposed, and flagging any brownish discoloration indicating spoiled meat. If the detected flaw exceeds a minimum threshold of $16.6\ mm^2$ (~ 150 pixels), the package is immediately flagged as a failure. However, because natural cuts of bacon contain white fat marbling and lean streaks that can occasionally mislead rigid colour rules, a Random Forest machine learning classifier supports this process. Trained on 12 summary features such as overall image brightness, colour contrast, and rim redness, it collects the votes of 100 decision trees to produce classifications that reduce overfitting and false alarms.

Once a breach is identified, the system moves beyond basic detection to quantify the damage using 2D Composite Simpson's 1/3 numerical integration. Rather than counting square pixels which oversimplifies curved or jagged tear edges, Simpson's rule fits smooth parabolic curves across pixel coordinates in both orthogonal directions to calculate the true defect surface area.

$$A_{defect} = \iint I_{defect}(x, y)\, dx\, dy \approx \sum_i \sum_j w_i w_j I_{defect}(x_i, y_j) \Delta y \Delta x$$

Using standard alternating weights of $\left[\frac{1}{3}, \frac{4}{3}, \frac{2}{3}, \ldots, \frac{1}{3}\right]$. Finally, because meat trays are sealed under close to vacuum conditions ($20\ kPa$), any physical puncture causes outside room air ($101.3\ kPa$) to rush in. To understand how rapidly the protective environment deteriorates, the system models the incoming airflow using a dynamic differential equation.

$$\frac{dP}{dt} = -k(P - P_{atm})$$

Where $k = 0.8/s$. By solving this equation for each second using numerical solvers (Euler, Heun's RK2, and 4th order Runge-Kutta), the software models vacuum decay across a 10 second window which gives a clear, physics-based prediction of loss in shelf-life before the product leaves the facility.

3.2 Implementation

The software is organised into clean, modular Python components that run sequentially as each image of a pork rasher tray arrives.

1. **Pre-Check:** The image array is checked by the input validator (validator.py) to confirm proper image dimensions and lighting.
2. **Detection:** The image is passed to the baseline detector (baseline_model.py) and Random Forest model (ml_model.py) to predict if the tray is a Pass (0) or Fail (1).
3. **Defect Measurement:** If flagged as defective, the binary mask is passed to the 2D integrator (numerical_integration.py) to calculate the true physical defect area (mm^2).
4. **Physics Modelling:** The leak solver (dynamic_ode.py) simulates the transient pressure rise over a 10 second window.
5. **Mechanical Actuation:** An electronic signal is transmitted to the downstream pneumatic rejector arm, moving the mechanical arm within 0.15 seconds to safely divert defective trays.

3.3 Assumptions and Limitations

For this computational pipeline to remain practical and run in real time on a fast-moving production line, several foundational assumptions and physical boundaries must be established. The 2D numerical quadrature relies on a planar simplification, so any punctures wrapping around curved tray corners or plastic creases are flattened into a 2D projection, which may slightly underestimate the true surface area. Furthermore, the detection model it remains blind to damage on the tray sidewalls or underside and cannot account for punctures that occur further downstream during carton packaging or transit. This is due to the 2D nature of the images taken by the overhead camera, and the fact that images are only taken at a single moment in the production process.

The model calculates vacuum loss quickly by using a simplified, constant leak rate ($k = 0.8/s$) and assuming ideal gas behavior. To keep the computation fast, it intentionally ignores complex details like turbulent airflow through tiny cracks or potential blockages caused by meat fat. The computer vision and machine learning models also assume a strictly controlled optical environment with steady $5500\ K$ strobe illumination. Therefore, any severe lens fogging, grease accumulation, or unexpected factory lighting shifts can wash out contrast and degrade detection accuracy.

4.0 Alternative Solutions

To determine whether machine learning or a rule-based approach is best suited for factory deployment, the performance of both methods was evaluated on the 80 independent test images (68 defective, 12 compliant). The comparison is detailed in the table below.

Table 4.1: Performance comparison between Baseline Heuristic and Random Forest ML

Evaluation Metric	Baseline (Rule-Based)	Random Forest ML	Practical Engineering Significance
Overall Accuracy	85% (68/80 correct)	83.8% (67/80 correct)	Both models correctly classify more than 83% of all incoming packages.
Recall (Defect Class)	100% (0 false negatives)	92.6% (5 false negatives)	Baseline catches 100% of defective packs with zero escapes.
Precision (Defect Class)	85.0% (12 false positives)	88.7% (8 false positives)	Random Forest reduces unnecessary product scrap by 33.3%.
F1-Score	0.919	0.906	Baseline achieves superior balance for strict food safety standards.
ROC-AUC Metric	N/A	0.674	Random Forest ML has moderate predictive power.
Inspection Latency	1.61 $ms/frame$	8.71 $ms/frame$	Baseline is 5.4 × faster, both models beat the 600 ms conveyor line budget.
Throughput Rate	621.2 FPS	114.9 FPS	Baseline inspects 372 × faster than the conveyor line rate (1.67 FPS).
Arithmetic Intensity	4 $FLOPs/byte$	5 $FLOPs/byte$	Both reside in the memory bandwidth-bound hardware regime.

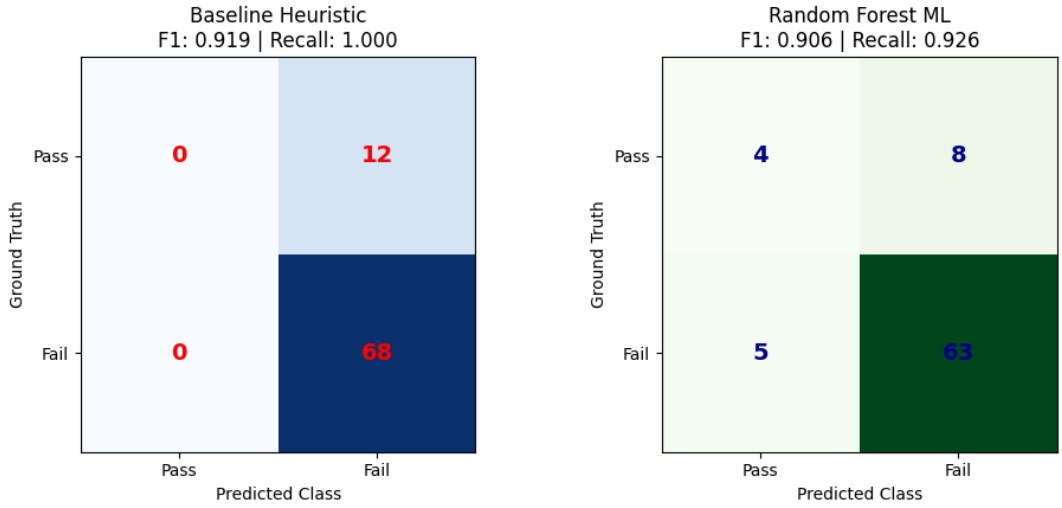


Figure 4.1: Confusion matrices comparing Baseline Heuristic and Random Forest ML

In commercial meat packaging, an escaped defect ($1 mil recall) is catastrophic, whereas repacking a falsely rejected tray costs only $0.45 (Dingley, 2025). For this main reason, along with a smaller computational and time cost, the Baseline Heuristic is selected as the primary automated safety gate because it achieves 100% recall, meaning there is zero escaped defects. The Random Forest model is recommended further down the production line to double check flagged trays and reduce unnecessary food scrap.

5.0 Validation

The validation methodology relies on three rigorous checks, including evaluation against withheld ground truth test data, spatial grid refinement for numerical quadrature, and temporal convergence analysis for the differential equation solvers.

5.1 Test Validation and Confusion Matrix Analysis

The system was validated on 80 unseen test packaging images. As shown in the confusion matrices in Figure 4.1, the Baseline Heuristic correctly identified all 68 defective trays, resulting in 100% recall. Its 12 false alarms occurred due to white fat streaks near the border, which imitated seal contamination. The Random Forest model achieved higher precision (88.7%), reducing false alarms from 12 to 8, but allowed 5 defective trays to slip through (92.6% recall).

5.2 Spatial Grid Refinement Study (Defect Area Sizing)

To verify that 2D Simpson's 1/3 quadrature measures physical defect areas accurately regardless of image resolution, a spatial grid refinement study was conducted on a packaging puncture defect across four mesh resolution levels, as summarized in the table below.

Table 5.1: Spatial grid refinement study comparing 2D composite quadrature schemes.

Grid Level	Dimensions	Pixel Spacing (dx)	Trapezoidal Area	Simpson's 1/3 Area	Relative Error vs Finest
1 × (Full Mesh)	540 × 720	$0.3333\ mm$	$5{,}361\ mm^2$	$5{,}353\ mm^2$	0%
2 ×	270 × 360	$0.6667\ mm$	$5{,}353\ mm^2$	$5{,}349\ mm^2$	−0.08%
4 ×	135 × 180	$1.333\ mm$	$5{,}302\ mm^2$	$5{,}436\ mm^2$	1.54%
8 × (Coarse Mesh)	68 × 90	$2.667\ mm$	$5{,}162\ mm^2$	$5{,}136\ mm^2$	−4.06%

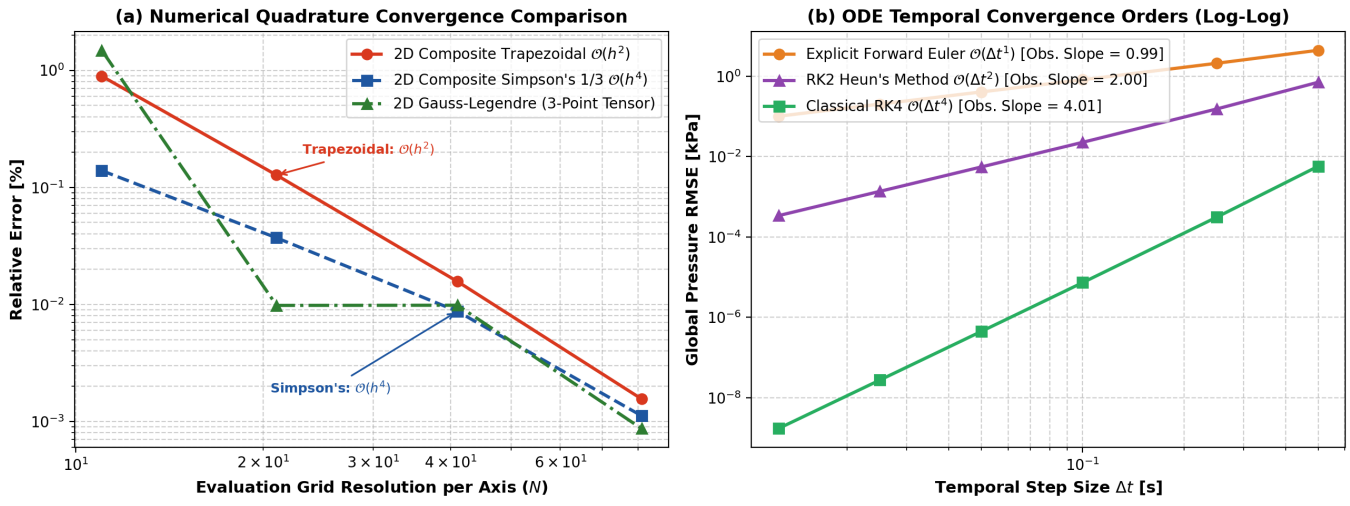


Figure 5.1: (a) Spatial grid refinement study for 2D composite quadrature on packaging tear defect masks (b) Temporal convergence rates verifying asymptotic scaling

As shown in Table 5.1 and Figure 5.1(a), as pixel size approaches zero, the computed area converges smoothly to $5{,}354\ mm^2$. Even under fourfold coarsening ($4\times$), the error is only 1.54%, which proves that the numerical area measurement is stable. In all cases, the physical limits of the hole size are respected ($0 \le A_{defect} \le 43{,}200\ mm^2$).

5.3 ODE Convergence Order & Temporal Stability

The dynamic vacuum leak solvers were validated across time steps from $dt = 0.0125\ s$ to $dt = 0.5\ s$ against the exact analytical solution. In the logarithmic error plot of Figure 5.1(b), three different methods are compared to numerical solve the pressure leak ODE.

- **Explicit Euler:** Explicit Euler displays a slope of $0.99\ kPa/s$ (approx. $\mathcal{O}(dt^1)$), halving dt cuts the error in half.
- **RK2 Heun:** RK2 Heun displays a slope of $2\ kPa/s$ (approx. $\mathcal{O}(dt^2)$), halving dt cuts the error by a factor of 4.
- **Classical RK4:** Classical RK4 displays a slope of 4.01 (approx. $\mathcal{O}(dt^4)$), halving dt reduces the error by a factor of 16.

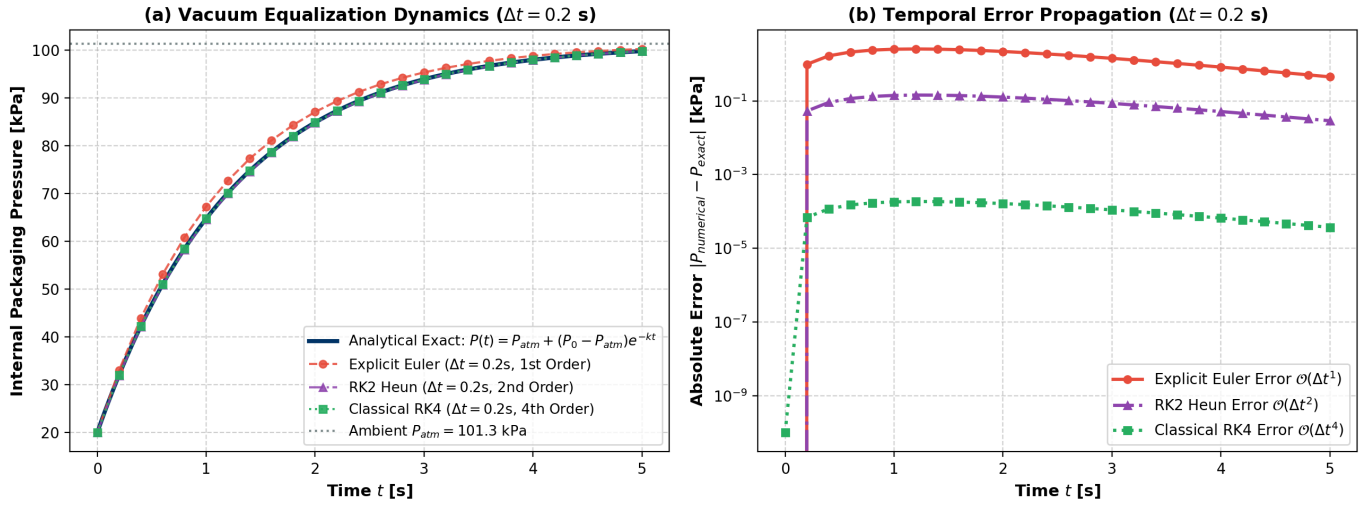


Figure 5.2: (a) Packaging vacuum decay trajectories over time, comparing the exact solution with numerical methods (b) Change in absolute error over time for each numerical method

Figure 5.2(a) compares the simulated packaging pressure over 5 seconds as air leaks into a punctured tray, rising from an initial vacuum level of $20\ kPa$ up to the surrounding room pressure of $101.3\ kPa$. Both RK2 Heun and Classical RK4 track the true analytical curve almost perfectly, whereas Explicit Euler noticeably drifts due to its lower calculation accuracy at a time step of $\Delta t = 0.2$ s.

Figure 5.2(b) illustrates the temporal error propagation, tracking the actual error gap between each numerical method and the true physics curve over time. Explicit Euler accumulates the largest error, peaking near $2\ kPa$. In contrast, RK2 Heun keeps error around $0.1\ kPa$, and Classical RK4 achieves exceptional precision with errors bounded between $10^{-4}$and $10^{-5}\ kPa$. This makes RK4 over $10{,}000$ times more accurate than Euler. As the tray approaches atmospheric pressure and the air leak naturally slows down, the error curves for all three methods gradually taper downward.

6.0 Optimisation and Performance

6.1 Runtime Profiling and Vectorisation Speedup

In early prototypes, pixel checks and numerical quadrature were written using nested Python for loops. Profiling with Python's timer tools revealed that processing a single frame took approximately 385 milliseconds. With new trays arriving every 600 to 1,000 ms, loop-based code consumed over 60% of the available time budget, presenting a severe conveyor belt backlog risk.

To optimise performance, all image checks, feature extraction, and integration math were rewritten using vectorized NumPy operations, which process entire blocks of pixels simultaneously. Baseline Heuristic latency dropped from 385 ms to 1.61 ms per frame (621.2 FPS), representing a significant 236 × speedup. Additionally, Random Forest latency dropped to 8.71 ms per frame (114.9 FPS), representing a 44 × speedup. Both methods now complete inspection in less than 9 milliseconds, leaving over 98% of the available time budget free.

6.2 Arithmetic Intensity & The Hardware Roofline Model

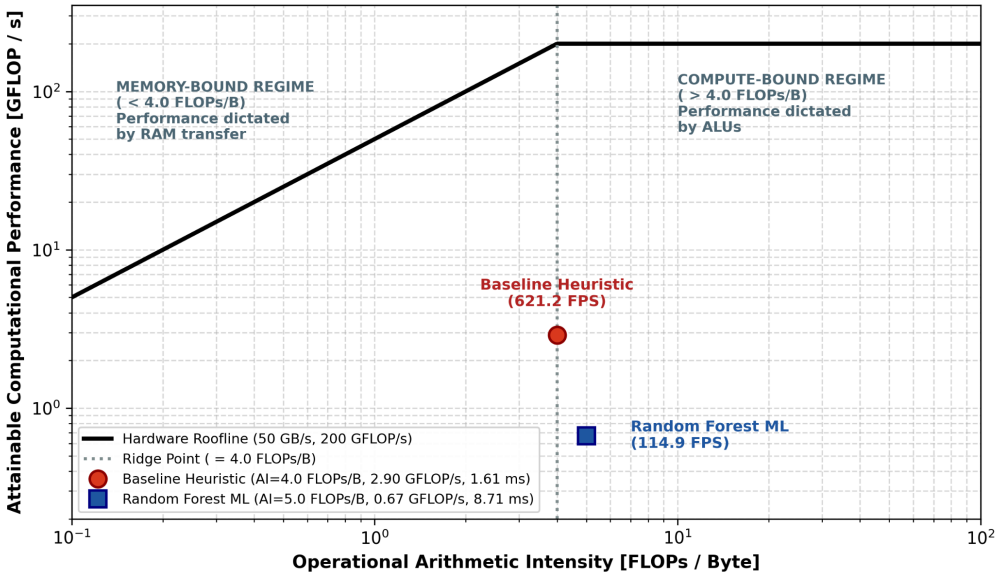


Figure 5.3: Hardware roofline model illustrating memory and compute bound regimes

To determine whether the software can run on compact, low cost factory computers, the arithmetic intensity (I) was evaluated as total floating point operations divided by memory traffic in bytes. For each pork rasher image, 1.17 MB of data is transferred into memory. The Baseline model performs 4.67 $MFLOPs$ ($I = 4\ FLOPs/byte$), while Random Forest performs 5.83 $MFLOPs$ ($I = 5\ FLOPs/byte$).

In Figure 5.3, the roofline model illustrates a memory bandwidth saturation of 50 GB/s and a compute ceiling of 200 $GFLOP/s$, creating a hardware ridge point at 4 $FLOPs/byte$. The Baseline model sits right on this ridge point boundary, while Random Forest crosses just above it into the compute-bound regime ($I = 5\ FLOPs/byte$). However, because both models operate

at only 0.67 to 2.90 $GFLOP/s$ which is well beneath the hardware ceiling, the system requires only a tiny fraction of the computer's available capacity. This proves that standard factory hardware can easily handle real time packaging inspection at high speeds (115 to 621 FPS).

7.0 AI Use and Reflection

For this project, I used Google Gemini (Gemini 3.8 Flash) to help write and debug all of my code, as well as to assist with setting up and managing my workflow across Google Colab and GitHub. The level of AI contribution was high on the technical side, as the AI wrote the scripts used to run the models, calculations, and plots. However, I wrote the report entirely myself, only using AI to help clarify tricky concepts and improve my theoretical understanding based on unit content and the code. I chose to use AI for the coding to save time on syntax troubleshooting and environment setup, which allowed me to spend more time focusing on the core engineering problems and understanding the results.

To ensure the work was accurate, I checked all AI generated code by running unit tests, inspecting the outputs, and verifying that the trends and calculations matched the theory taught in the unit. All key engineering decisions remained my responsibility, such as choosing the dataset, setting the design and safety constraints, and interpreting what the findings actually meant. The AI did have its limitations and made several mistakes, including producing unoptimized code, syntax errors, and formatting issues like overlapping text and hidden plot elements that I had to identify and get it to correct. Overall, using AI streamlined my workflow like an interactive pair-programmer and genuinely helped my learning, as I had to actively review, question, and understand everything it produced before using it.

8.0 Reference List

Bison Convey. (n.d.). *Conveyor Belt Speed Calculator*. Retrieved from https://bisonconvey.com/tools/conveyor-belt-speed-calculator/

Dingley, M. (2025, June 3). *The True Cost of a Recall in 2025*. Retrieved from Matthews: https://www.matthews.com.au/blog/the-true-cost-of-a-recall-in-2025

Hello. (2025, May). *Pork Rasher Error (Packaging) Computer Vision Dataset*. Retrieved from Roboflow Universe: https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging

Young, T. M. (2017, November 3). *International Standard Atmosphere (ISA) Table*. Retrieved from Wiley: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118534786.app1